



OPEN
Compute Project

Hardware Fault Management Project

System Debug Specification

0.1

Modular Base Specification
Effective April 30, 2026

Author: (See Acknowledgements section)

Current Template Version:

3 Layer (Base, Design and Product) Specification Template V1.6.0

Table of Contents

1 License	6
2 Acknowledgements	7
3 Revision History	8
4 Compliance with OCP Tenets	9
4.1 Openness	9
4.2 Efficiency	9
4.3 Impact	9
4.4 Scale	9
4.5 Sustainability	9
5 Introduction	10
5.1 Goals	10
5.2 Document Scope	10
5.2.1 In Scope	10
5.2.2 Out of Scope	10
5.3 Purpose	10
5.4 Audience	10
6 Terminology	11
6.1 Syntax and Conventions	11
6.1.1 Special Terms	11
6.1.2 Number Radix	11
6.2 Definitions	11
6.3 Abbreviations	11
6.4 Acronyms	11
7 Technical Overview (Informative)	13
8 Layering	14
9 Handling Multiple Users	15
10 Security and Privacy	16
11 Data Stacks	17
11.1 Code Currency	17
11.1.1 Schema	17
11.1.2 Field Explanations	20
11.1.3 Example	21
11.1.4 Code Currency Redfish Schema	26
11.2 Status Dump	26
11.3 Console	26
11.4 Interactive Debug/Run Control	26
12 Future Work	27
Appendices	28
A First appendix	28
B Second appendix	29
References	30

List of Figures

1	Product Development Life Cycle	13
2	Execution Flow	13
3	Code Currency Relationships	20
4	Example Code Currency	25

List of Tables

1 License

Open Web Foundation (OWF) CLA

Contributions to this Specification are made under the terms and conditions set forth in **Modified Open Web Foundation Agreement 0.9 (OWFa 0.9)**. (As of October 16, 2024) (“Contribution License”) by:

- TODO: fill in

Usage of this Specification is governed by the terms and conditions set forth in **Modified OWFa 0.9 Final Specification Agreement (FSA)** (As of October 16, 2024) (“**Specification License**”).

You can review the applicable Specification License(s) referenced above by the contributors to this Specification on the OCP website at <https://www.opencompute.org/contributions/templates-agreements>.

For actual executed copies of either agreement, please contact OCP directly.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP “AS IS” AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY DAMAGES WITH RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

2 Acknowledgements

The Contributors of this Specification would like to acknowledge the following:

- Marko Bartscherer (Intel)
- Enrico Carrieri (Qualcomm Technologies, Inc.)

Also, we wish to acknowledge Chris Fenner (Google) and the Trusted Computing Group for the use of their Pandoc markdown tooling.

3 Revision History

Rev.	Date	Guiding Contributor(s)	Description
0.1	2026-04-29	Marko B., Enrico C.	Initial Release

4 Compliance with OCP Tenets

4.1 Openness

This specification is open.

4.2 Efficiency

This specification is efficient.

4.3 Impact

This specification is impactful.

4.4 Scale

This specification is scalable.

4.5 Sustainability

This specification is sustainable.

5 Introduction

With the rise of large datacenter and AI systems, nodes and components are becoming more diverse and increasing in size and complexity. This increasing complexity requires the need to validate, diagnose, debug, and optimize beyond the component level. Before the publication of this specification, the datacenter community was lacking a standardized methodology to extract diagnostic data and perform debug operations across the datacenter at the different levels from components to cluster. There was a need to develop a specification that defines how debug data is exchanged between all components across the datacenter and the debug tooling. The community desired to standardize the necessary control and transport protocols enabling uniform debug of diverse systems.

This specification will address debugging components and nodes within a datacenter across the lifecycle of the datacenter. It will prescribe an architecture to communicate various debug data in a common method and data models across the datacenter. The specification will define interfaces, protocols, APIs, and data models incorporating existing work from other OCP Workstreams and other standards bodies improving the effectiveness of these for debugging in the datacenter. Scalability across the lifecycle of the datacenter including but not limited to accessible ports, diagnostic and debug capabilities, and security and privacy requirements.

5.1 Goals

The main goal of this specification is to prescribe an architecture, including interfaces, protocols, APIs, and data models, that communicates debug data across the datacenter. This will be done by:

- Defining the use of common connectivity, transports, and protocols.
 - Define a common communication “pipe” for debug data/messages.
 - Data/message payloads stay vendor specific.
- Using existing specifications/standards when available.
 - Leverage existing standard transports (e.g., USB) and protocols (e.g., PLDM)
- Allowing flexibility but provide interoperability.
 - Flexibility in how the system is put together.
 - Debug capabilities need to be discoverable and self describing
 - Interoperability with both different components and different debug tooling.
 - Data needs to be opaque to components in the middle (e.g., BMC).
- Providing scalability across the life cycle of development.
 - Use starting from manufacturing through to the at-scale, in-field deployment.

5.2 Document Scope

5.2.1 In Scope

5.2.2 Out of Scope

This specification will not cover existing material already covered by another OCP specification even if it is related to the goals stated in Section [5.1](#).

5.3 Purpose

5.4 Audience

6 Terminology

6.1 Syntax and Conventions

6.1.1 Special Terms

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in [BCP 14] [RFC2119] [RFC8174] when they appear.

6.1.2 Number Radix

This document may represent some numbers in non-decimal radix. In these cases, the following radix notations are used:

4'b0001	Binary representation of the decimal value 1 explicitly expressed in 4 bits.
0x5A	Hexadecimal representation of the decimal value 90.
7'h5A	Hexadecimal representation of the decimal value 90 explicitly expressed in 7 bits.

Note:

An underscore (i.e., ‘_’) is sometimes used in the representation to improve readability. For example, an underscore can be used in a Hexadecimal number to separate each four digits in the example: 0x0001_2345.

6.2 Definitions

- **Hyperscalers** - Cloud service providers of the world, who value scaling and rapid time to market of new hardware products.
- **Life Cycle** -
- **Node** - A platform designed by a supplier integrating multiple components that run an operating system. These are sometimes also called compute nodes.

6.3 Abbreviations

DBG	Debug
-----	-------

6.4 Acronyms

BMC	Baseboard Management Controller
CPER	Common Platform Error Record
FW	Firmware
JSON	JavaScript Object Notation
JTAG	Joint Test Action Group - Bare metal interface to access on-chip debug and test resources.
MTTD	Mean Time to Debug

(continued on next page)

(continued from previous page)

OCP	Open Compute Project
PLDM	Platform Level Data Model
RAS	Reliability, Availability, and Serviceability
SW	Software
WDT	Watchdog Timer

7 Technical Overview (Informative)

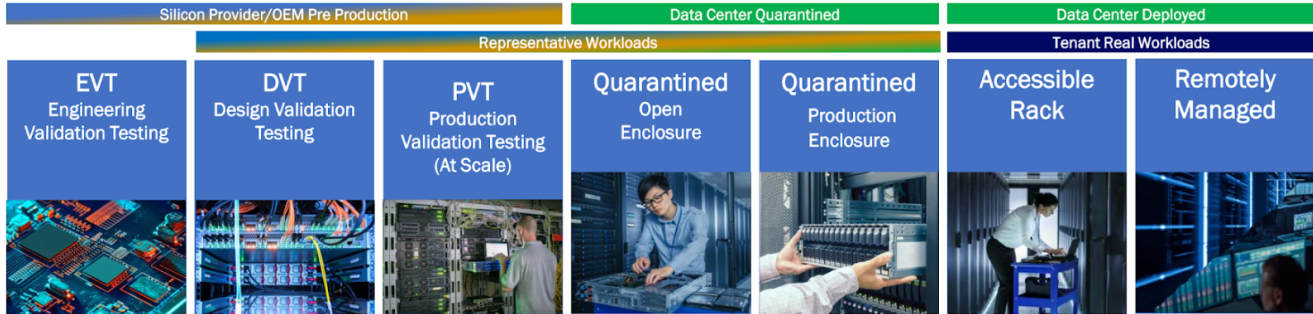


Figure 1: Product Development Life Cycle

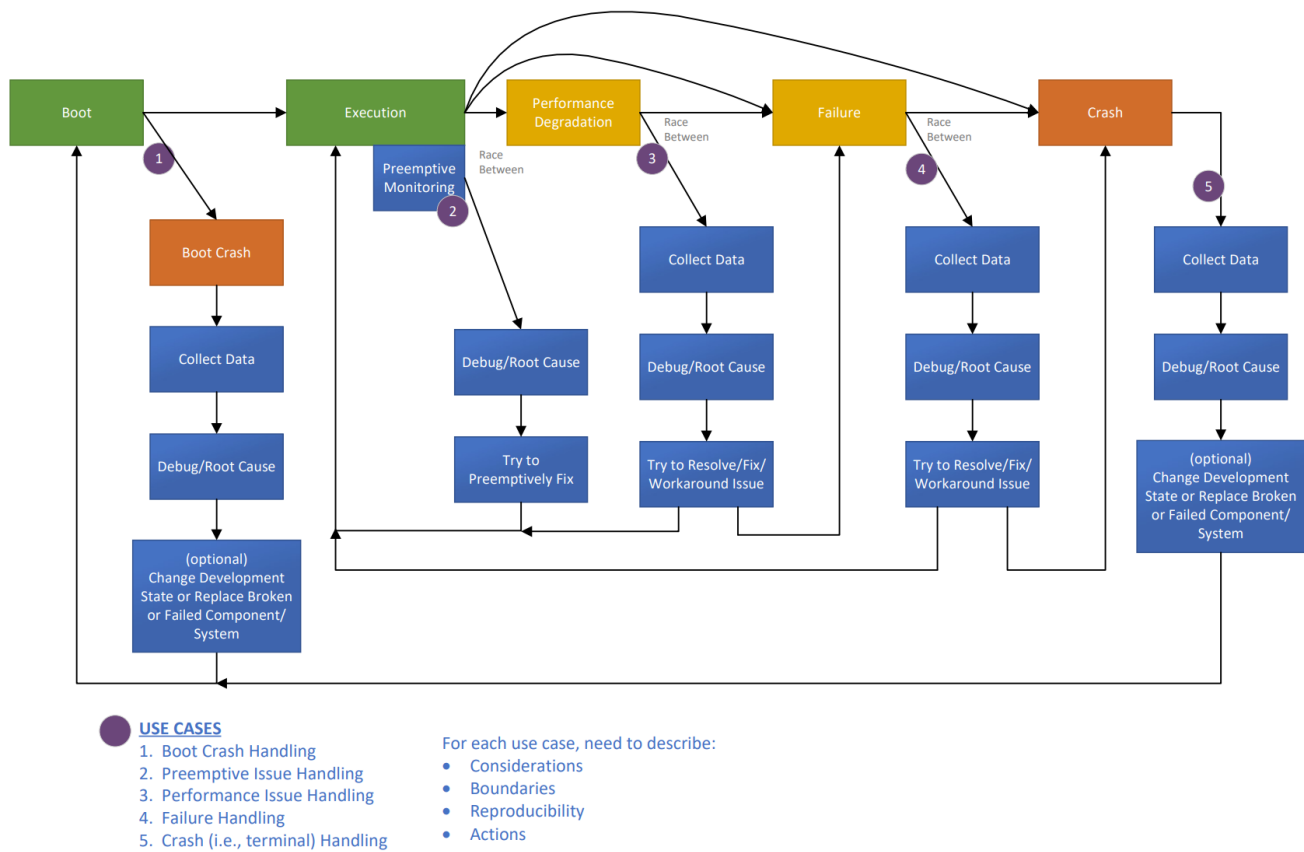


Figure 2: Execution Flow

8 Layering

9 Handling Multiple Users

10 Security and Privacy

11 Data Stacks

11.1 Code Currency

More often than not the root cause of a failure or problem is simply that the system has not been configured correctly, it is run either outdated FW, an incompatible combination of SW/FW components or components that have not been configured appropriately. The concept of Code Currency has been designed to address this issue by providing a one-stop shop style standardized machine readable method to retrieve all relevant information about the configuration of a given system. The information will be exposed by a system as a file in the Redfish profile of the system. Components and subsystem will expose their part of the information to the Main BMC as PLDM type 7 files. The content of the file is organized using the JSON schema outlined in this file.

11.1.1 Schema

This section defines the **JSON schema** for representing the firmware, configuration, and physical/logical details of nodes, assemblies, and components in a datacenter environment.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Node Code Currency Record",
  "type": "object",
  "properties": {
    "NodeId": { "type": "string", "description": "Unique identifier for the node" },
    "NodeType": { "type": "string", "description": "Type of node (e.g., AI Server, Ethernet Switch, Management Tray)" },
    "Hostname": { "type": "string", "description": "Node hostname" },
    "NetworkConfiguration": {
      "type": "object",
      "description": "Network configuration details for the node",
      "properties": {
        "IpAddresses": { "type": "array", "items": { "type": "string", "format": "ipv4" } },
        "Ports": { "type": "array", "items": { "type": "integer", "minimum": 1, "maximum": 65535 } },
        "MacAddresses": { "type": "array", "items": { "type": "string", "pattern": "^[0-9A-Fa-f]{2:}{5}{0-9A-Fa-f}{2}$" } }
      },
      "required": ["IpAddresses"]
    },
    "PhysicalLocation": {
      "type": "object",
      "description": "Physical location of the node in the datacenter",
      "properties": {
        "Rack": { "type": "string" },
        "Row": { "type": "string" },
        "Datacenter": { "type": "string" },
        "Placement": {
          "type": "object",
          "properties": {
            "Type": { "type": "string", "enum": ["RackSlot", "Bay"] },
            "Slot": { "type": "integer", "description": "Rack unit position if RackSlot" },
            "HeightInU": { "type": "integer", "description": "Height in U if RackSlot" },
            "BayId": { "type": "string", "description": "Bay identifier if Bay" },
            "BayPosition": { "type": "string", "description": "Position within the bay" }
          },
          "required": ["Type"]
        }
      },
      "required": ["Rack", "Row", "Datacenter", "Placement"]
    },
    "Assemblies": {
      "type": "array",
      "description": "List of assemblies within the node (supports nested assemblies)",

```

```

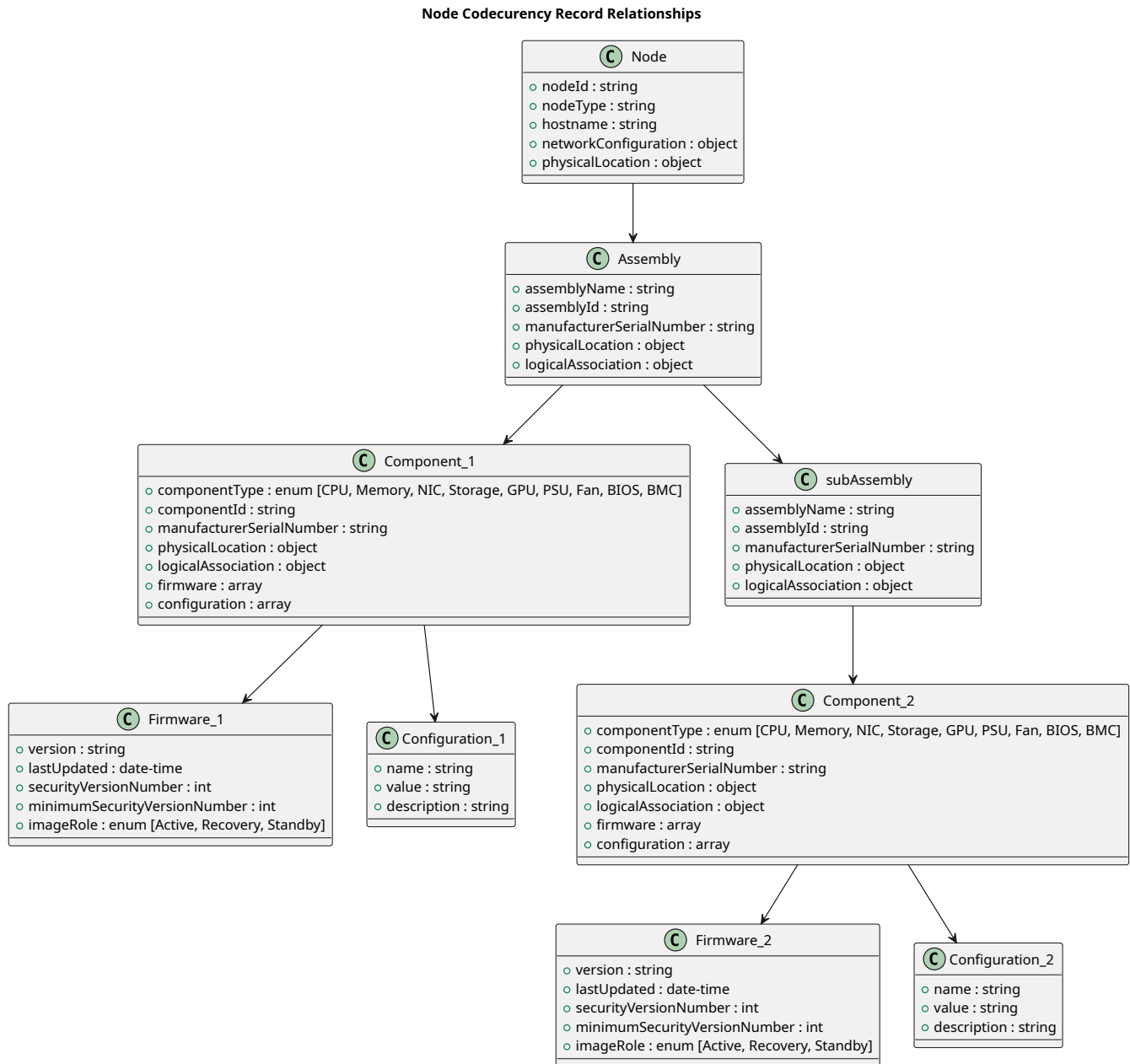
        "items": { "$ref": "#/$defs/Assembly" }
    },
    "required": ["NodeId", "NodeType", "Assemblies"],
    "$defs": {
        "Assembly": {
            "type": "object",
            "properties": {
                "AssemblyName": { "type": "string" },
                "AssemblyId": { "type": "string" },
                "ManufacturerSerialNumber": { "type": "string", "description": "Manufacturer's serial
↪ number for the assembly" },
                "PhysicalLocation": {
                    "type": "object",
                    "description": "Physical slot or bay where this assembly is installed",
                    "properties": {
                        "Slot": { "type": "string", "description": "Physical slot identifier (e.g., PCIe slot,
↪ OCP slot)" },
                        "Position": { "type": "string", "description": "Position within the node or assembly" }
                    }
                },
                "LogicalAssociation": {
                    "type": "object",
                    "description": "Logical connectivity details for this assembly",
                    "properties": {
                        "Links": {
                            "type": "array",
                            "description": "List of logical links consumed by this assembly",
                            "items": {
                                "type": "object",
                                "properties": {
                                    "ConnectedTo": { "type": "string" },
                                    "Protocol": { "type": "string" },
                                    "LaneCount": { "type": "integer" }
                                },
                                "required": ["ConnectedTo", "Protocol"]
                            }
                        }
                    }
                },
                "Components": { "type": "array", "items": { "$ref": "#/$defs/Component" } },
                "SubAssemblies": { "type": "array", "items": { "$ref": "#/$defs/Assembly" } }
            },
            "required": ["AssemblyName", "AssemblyId"]
        },
        "Component": {
            "type": "object",
            "properties": {
                "ComponentType": { "type": "string", "enum": ["CPU", "Memory", "NIC", "Storage", "GPU",
↪ "PSU", "Fan", "BIOS", "BMC", "AMC"] },
                "ComponentId": { "type": "string" },
                "ManufacturerSerialNumber": { "type": "string", "description": "Manufacturer's serial
↪ number for the component" },
                "PhysicalLocation": {
                    "type": "object",
                    "description": "Physical slot or bay where this component is installed",
                    "properties": {
                        "Slot": { "type": "string", "description": "Physical slot identifier (e.g., PCIe slot,
↪ DIMM slot)" },
                        "Position": { "type": "string", "description": "Position within the node or assembly" }
                    }
                },
                "LogicalAssociation": {
                    "type": "object",
                    "description": "Logical connectivity details for this component",
                    "properties": {
                        "Links": {
                            "type": "array",

```

```

        "description": "List of logical links consumed by this component",
        "items": {
            "type": "object",
            "properties": {
                "ConnectedTo": { "type": "string" },
                "Protocol": { "type": "string" },
                "LaneCount": { "type": "integer" }
            },
            "required": ["ConnectedTo", "Protocol"]
        }
    },
    "Firmware": {
        "type": "array",
        "description": "List of firmware entries for the component",
        "items": {
            "type": "object",
            "properties": {
                "Version": { "type": "string", "description": "Firmware version" },
                "LastUpdated": { "type": "string", "format": "date-time", "description": "Timestamp
                    ↳ of last firmware update" },
                "SecurityVersionNumber": { "type": "integer", "description": "Current Security
                    ↳ Version Number (SVN)" },
                "MinimumSecurityVersionNumber": { "type": "integer", "description": "Minimum enforced
                    ↳ Security Version Number (SVN)" },
                "ImageRole": {
                    "type": "string",
                    "enum": ["Active", "Recovery", "Standby"],
                    "description": "Indicates if this firmware is actively running or a redundant
                        ↳ recovery image"
                }
            },
            "required": ["Version"]
        }
    },
    "Configuration": {
        "type": "array",
        "description": "Custom configuration attributes for the component",
        "items": {
            "type": "object",
            "properties": {
                "Name": { "type": "string", "description": "Configuration attribute name" },
                "Value": { "type": "string", "description": "Configuration attribute value" },
                "Description": { "type": "string", "description": "Optional description of the
                    ↳ attribute" }
            },
            "required": ["Name", "Value"]
        }
    },
    "required": ["ComponentType", "ComponentId", "Firmware"]
}

```

**Figure 3: Code Currency Relationships**

11.1.2 Field Explanations

11.1.2.1 Node-Level

- nodeId: Unique identifier for the node.
- nodeType: Type of node (e.g., AI Server, Ethernet Switch).
- hostname: Hostname for network identification.
- networkConfiguration:
 - ipAddresses: List of IPv4 addresses.

- ports: Open or configured ports.
- macAddresses: MAC addresses for network interfaces.
- physicalLocation:
 - rack, row, datacenter: Physical placement identifiers.
 - placement:
 - * type: RackSlot or Bay.
 - * slot: Rack unit position.
 - * heightInU: Height in rack units.
 - * bayId, bayPosition: Bay-specific identifiers.

11.1.2.2 Assemblies

- assemblyName: Name of the assembly (e.g., Baseboard).
- assemblyId: Unique identifier.
- manufacturerSerialNumber: Manufacturer's serial number.
- physicalLocation: Slot and position within node.
- logicalAssociation:
 - links: Array of logical connections (connectedTo, protocol, laneCount).
- components: Array of components.
- subAssemblies: Nested assemblies.

11.1.2.3 Components

- componentType: Type (CPU, Memory, NIC, etc.).
- componentId: Unique identifier.
- manufacturerSerialNumber: Manufacturer's serial number.
- physicalLocation: Slot and position.
- logicalAssociation: Logical links consumed by the component.
- firmware:
 - version: Firmware version (required).
 - lastUpdated: Timestamp of last update.
 - securityVersionNumber: Current SVN.
 - minimumSecurityVersionNumber: Minimum enforced SVN.
 - imageRole: Active, Recovery, or Standby.
- configuration:
 - Array of custom attributes: name, value, optional description.

11.1.3 Example

Below is an example of an Hypothetical Node with:

- Dual-socket CPUs
- Memory modules
- BMC
- Two NVMe drives
- Two NICs on a hot-swap backplane

- UBB subassembly with BMC
- Two OAM subassemblies (each with GPU and AMC)

```
{
  "NodeId": "AI-Node-001",
  "NodeType": "AI Server",
  "Hostname": "ai-node-001.dc.local",
  "NetworkConfiguration": {
    "IpAddresses": ["192.168.10.101"],
    "Ports": [22, 443],
    "MacAddresses": ["00:1A:2B:3C:4D:5E"]
  },
  "PhysicalLocation": {
    "Rack": "R42",
    "Row": "Row-A",
    "Datacenter": "DC-West",
    "Placement": {
      "Type": "RackSlot",
      "Slot": 12,
      "HeightInU": 2
    }
  },
  "Assemblies": [
    {
      "AssemblyName": "Baseboard",
      "AssemblyId": "BB-001",
      "ManufacturerSerialNumber": "BB-SN-12345",
      "Components": [
        {
          "ComponentType": "CPU",
          "ComponentId": "CPU-001",
          "ManufacturerSerialNumber": "CPU-SN-11111",
          "Firmware": [
            {
              "Version": "1.2.3",
              "LastUpdated": "2025-10-01T12:00:00Z",
              "SecurityVersionNumber": 5,
              "MinimumSecurityVersionNumber": 3,
              "ImageRole": "Active"
            }
          ]
        },
        {
          "ComponentType": "CPU",
          "ComponentId": "CPU-002",
          "ManufacturerSerialNumber": "CPU-SN-22222",
          "Firmware": [
            {
              "Version": "1.2.3",
              "LastUpdated": "2025-10-01T12:00:00Z",
              "SecurityVersionNumber": 5,
              "MinimumSecurityVersionNumber": 3,
              "ImageRole": "Active"
            }
          ]
        }
      ]
    },
    {
      "ComponentType": "Memory",
      "ComponentId": "MEM-001",
      "ManufacturerSerialNumber": "MEM-SN-33333",
      "Firmware": [
        { "Version": "N/A" }
      ]
    },
    {
      "ComponentType": "Memory",
      "ComponentId": "MEM-002",
    }
  ]
}
```

```

    "ManufacturerSerialNumber": "MEM-SN-44444",
    "Firmware": [
      { "Version": "N/A" }
    ]
  },
  {
    "ComponentType": "BMC",
    "ComponentId": "BMC-001",
    "ManufacturerSerialNumber": "BMC-SN-55555",
    "Firmware": [
      {
        "Version": "3.4.5",
        "LastUpdated": "2025-09-15T08:00:00Z",
        "SecurityVersionNumber": 2,
        "MinimumSecurityVersionNumber": 1,
        "ImageRole": "Active"
      }
    ]
  }
],
"SubAssemblies": [
  {
    "AssemblyName": "HotSwap Backplane",
    "AssemblyId": "HSBP-001",
    "ManufacturerSerialNumber": "HSBP-SN-66666",
    "Components": [
      {
        "ComponentType": "Storage",
        "ComponentId": "NVME-001",
        "ManufacturerSerialNumber": "NVME-SN-77777",
        "Firmware": [
          { "Version": "2.1.0" }
        ]
      },
      {
        "ComponentType": "Storage",
        "ComponentId": "NVME-002",
        "ManufacturerSerialNumber": "NVME-SN-88888",
        "Firmware": [
          { "Version": "2.1.0" }
        ]
      },
      {
        "ComponentType": "NIC",
        "ComponentId": "NIC-001",
        "ManufacturerSerialNumber": "NIC-SN-99999",
        "Firmware": [
          { "Version": "1.0.0" }
        ]
      },
      {
        "ComponentType": "NIC",
        "ComponentId": "NIC-002",
        "ManufacturerSerialNumber": "NIC-SN-10101",
        "Firmware": [
          { "Version": "1.0.0" }
        ]
      }
    ]
  },
  {
    "AssemblyName": "UBB",
    "AssemblyId": "UBB-001",
    "ManufacturerSerialNumber": "UBB-SN-20202",
    "SubAssemblies": [
      {
        "AssemblyName": "OAM-1",
        "AssemblyId": "OAM-001",

```

```

    "ManufacturerSerialNumber": "OAM-SN-30303",
    "Components": [
      {
        "ComponentType": "GPU",
        "ComponentId": "GPU-001",
        "ManufacturerSerialNumber": "GPU-SN-40404",
        "Firmware": [
          { "Version": "510.85" }
        ]
      },
      {
        "ComponentType": "AMC",
        "ComponentId": "AMC-001",
        "ManufacturerSerialNumber": "AMC-SN-50505",
        "Firmware": [
          { "Version": "1.0.0" }
        ]
      }
    ]
  },
  {
    "AssemblyName": "OAM-2",
    "AssemblyId": "OAM-002",
    "ManufacturerSerialNumber": "OAM-SN-60606",
    "Components": [
      {
        "ComponentType": "GPU",
        "ComponentId": "GPU-002",
        "ManufacturerSerialNumber": "GPU-SN-70707",
        "Firmware": [
          { "Version": "510.85" }
        ]
      },
      {
        "ComponentType": "AMC",
        "ComponentId": "AMC-002",
        "ManufacturerSerialNumber": "AMC-SN-80808",
        "Firmware": [
          { "Version": "1.0.0" }
        ]
      }
    ]
  }
]
}

```

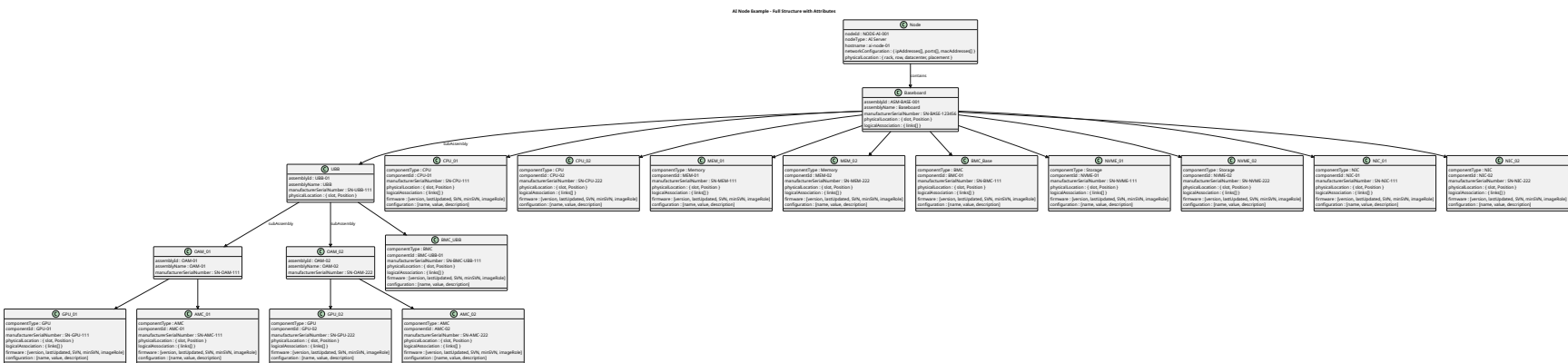


Figure 4: Example Code Currency

11.1.4 Code Currency Redfish Schema

11.2 Status Dump

11.3 Console

11.4 Interactive Debug/Run Control

12 Future Work

Appendix A: First appendix

This is the first appendix.

Appendix B: Second appendix

This is the second appendix.

References